** Content labeled as Internal, this info can only be shared within Vintecc! **

Secure Development Procedure

ISO27001

Change History:

Date	Version	Author	Description
14/10/2025	1.0	Pieterjan Cottignies	First Release

Definitions:

SSDLC: Secure Software development lifecycle

Contents

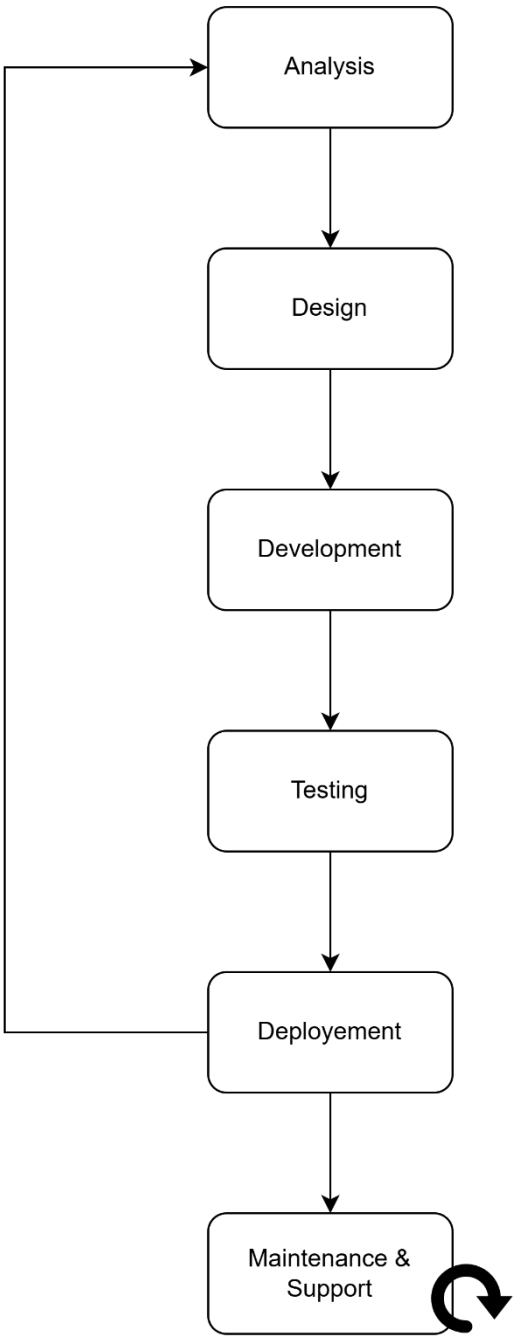
1. Introduction 3
2. Software development lifecycle flowchart 3
3. Secure Software Development steps 4
   3.1. Analysis 4
   3.2. Design 4
   3.3. Development 5
   3.4. Testing 5
   3.5. Deployment 6
   3.6. Maintenance & Support 6
   3.7. Team Responsibilities 7
4. Team Procedures 7
5. Documentation 7
6. References 8

1. Introduction

This procedure ensures security is embedded across all software development activities at Vintecc, covering internal and outsourced projects. It translates policy controls into operational steps, integrating SSDLC practices into daily development through structured phases.

It is intended to lay out a framework for teams to extend into their own procedures and guidelines providing a good basis for all teams within Vintecc to build on.

2. Software development lifecycle flowchart



3. Secure Software Development steps

In this chapter we aim to describe each step, why it's important and some examples or best practices we can incorporate in the specified step.

3.1. Analysis

Definition:
This step focuses on gathering and analyzing business needs and prioritizing security by including risk analysis, data classification, and legal or regulatory mapping. The goal is to ensure that every project starts with clarity about its security context, identifying assets, threats, and compliance obligations at the outset.

Musts (Policy):

- Perform a risk analysis and classify data; document compliance requirements.
- Specify business information sensitivity, asset value, and threat landscape.
- Ensure legal, regulatory, and contractual obligations are precisely described.
- Maintain a documented charter, outlining stakeholders and their responsibilities.

Recommendations:

- Use threat modeling templates and risk registers for consistency.
- Frequently review requirements and security with stakeholders and customers.

3.2. Design

Definition:
The design phase translates requirements into secure system architectures and components. It emphasizes threat modeling, secure-by-design principles, robust authentication, authorization, encryption strategies, and defensible architecture. Design reviews and selection of suitable patterns help minimize vulnerabilities before development starts.

Musts (Policy):

- Deliver documented secure architecture, considering access control and data protection.
- Conduct design reviews for security controls (validation, logging, recovery).
- Apply secure design principles such as secure key management and including cryptography where needed.

Recommendations:

- Model data flows and system interfaces visually.
- Use OWASP Application Security Verification Standard (ASVS) during review. Strive to achieve a minimum of level 2.
- Involve peer design review sessions for challenging components.

3.3. Development

Definition:
During development, code is written according to secure coding standards, using version control and strict access controls. This step enforces best practices, such as using peer reviews, documentation, OWASP Top 10 defenses, and automated security checks, to prevent common coding flaws and unauthorized changes.

Musts (Policy):
- Write code following approved secure coding standards, language-appropriate guidelines, and document all changes.
- Use version control with branch protections; maintain separation between environments (e.g. separate develop & production environment).
- Prohibit hard-coded credentials and enforce error handling.

Recommendations:
- Implement peer review and automated CI/CD security checks.
- Apply OWASP Top 10 and ASVS guidelines as practical reference.
- Keep an updated changelog and internal documentation.
- Encourage code scanning (SAST/DAST) tools in pipeline.
- Implement secret scanners to detect unwanted leaks of credentials or API-keys.

3.4. Testing

Definition:
This stage validates both functionality and security. Teams perform security testing (unit, integration, SAST, DAST, SBOM, and OWASP ASVS) to find and remediate vulnerabilities. Secure testing ensures that security controls work as intended and no critical risks are released with product builds.

Musts (Policy):
- Include security testing (unit, integration, SAST, DAST, SBOM, ASVS) before release.
- Maintain strict test data separation; anonymize sensitive information.

Recommendations:
- Define and share a test plan with acceptance criteria early.
- Mock APIs and seed databases for local testing fidelity.
- Cover all testing levels: unit, system, user, operational.
- Regular Penetration testing for important systems and components

3.5. Deployment

Definition:
Deployment automates and secures the release process using CI/CD setups with security scans, ensuring environment separation (development, test, production) and release approvals. The focus is on controlled, traceable releases, with mechanisms for rollback and incident response.

Musts (Policy):
- Only release via automated CI/CD pipelines with documented approval steps.
- Ensure strict separation and protection of all environments.
- Maintain deployment and rollback strategy documentation.
- Secret and key management for deployments e.g. Keeper, Azure Key Vault.

Recommendations:
- Automate post-deployment integrity checks in pipeline.
- Log all deployment actions for traceability.
- Include frequent communication with customer regarding updates.

3.6. Maintenance & Support

Definition:
Ongoing security activities, like patch management, vulnerability assessments, monitoring, and continuous updates, are essential. Maintenance ensures the system remains resilient against new vulnerabilities and that lessons learned from incidents feed into improvement.

Musts (Policy):
- Document and execute patching for all components; monitor vulnerabilities.
- Update and review operational guides and user documentation regularly.

Recommendations:
- Set regular maintenance windows and communicate to customers. Enforce these in contracts and clearly define them as requirements for SLA-agreements
- Automate vulnerability scanning, alerting and dependency updates if possible.
- Feed lessons learned from incidents and updates into continuous improvement.

3.7. Team Responsibilities

Musts (Policy):

- Assign a Security Champion in each team to drive SSDLC; developers must adhere to policy. This can coincide with Tech / Quality roles.
- Project leads ensure documentation and policy compliance.
- The security team supports, trains, and oversees secure practices.

Recommendations:

- Teams should define programming guidelines and checklist templates specific to their stack, updating procedures for technology changes.
- Hold regular cross-team security knowledge sharing sessions.

4. Team Procedures

This Secure Development Procedure provides a foundational framework for integrating security into software development. However, due to the diversity of technology stacks across teams, it does not prescribe specific technical implementations.

Each team is expected to **adapt this framework** into a tailored set of procedures that align with their tools, technologies, and workflows. All **mandatory controls ("Musts")** outlined in this procedure must be incorporated into the team's own development practices. Additionally, teams are encouraged to include relevant **recommendations** and any other security controls that enhance their secure development lifecycle.

The goal is to ensure that every team takes ownership of secure development by embedding these principles into their day-to-day operations.

5. Documentation

All key project documents, including the **risk analysis**, **project description**, and other required materials, must be stored in the project's designated **SharePoint location**. This ensures they are available as evidence of due diligence and can be referenced during:

- Recurring risk assessments
- Feature planning and development
- Security reviews and audits

Maintaining up-to-date documentation in SharePoint supports traceability, accountability, and continuous improvement across the secure development lifecycle.

6. References

Secure Development Template	Vintecc\Organization - Documents\Security\Policies & Procedures\Secure Development